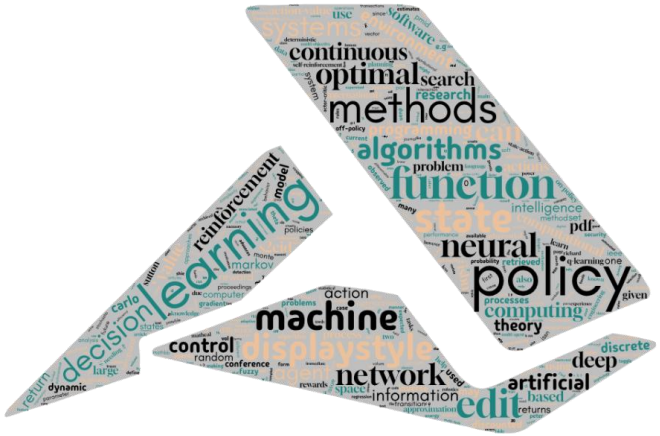


Reinforcement Learning Algorithms and Applications

Chapter 3: Monte Carlo for Black Jack

Introduction to Monte Carlo Learning

- **Monte Carlo methods** use **random sampling** for machine learning.
- Learn optimal strategies through **game simulations**.
- No prior knowledge of environment **dynamics** required.
- Uses **episode-based learning** from complete game experiences.
- Suitable for problems with **clear terminal states**.
- Improves strategies through **trial-and-error**.
- Handles **stochastic environments** effectively.

https://zh.wikipedia.org/zh-tw/%E4%BA%8C%E5%8D%81%E4%B8%80%E9%BB%9E

Blackjack Game Implementation

- **BlackjackGame class** simulates complete Blackjack environment.
- Creates **full deck** with four suits and thirteen ranks.
- **Ace cards** valued at 11, adjustable to 1.
- **Shuffling function** ensures game randomness.
- **Auto-reshuffle** when deck below 10 cards.
- **Hand value calculation** handles Ace dual values.
- **Bust detection** and **blackjack detection** included.

Monte Carlo Player Design

- **ImprovedMonteCarloPlayer class** as intelligent agent.
- **Q-table** stores state-action values.
- **Epsilon-greedy strategy** balances exploration/exploitation.
- **State space**: *(player_value, dealer_value, has_usable_ace)*.
- **Action space**: *(hit, stand)*.
- **Epsilon decay**: 0.9 → 0.05 gradually.
- **First-Visit Monte Carlo** for updates.
- **Basic strategy** as initial reference.

Training Process

episode = [
(state1, action1, reward1),
(state2, action2, reward2),
...]

- **200,000 episodes** of training simulation.
- Records **state-action-reward** sequences.
- **Reward**: win +1, loss -1, tie +0, natural blackjack +1.5.
- ***G***: The **final game outcome** in ONE episode, meaning the **total cumulative reward**
  - $G_t = R_{t+1} + R_{t+2} + \cdots + R_T$
  - R = reward
  - t = current time step in the episode
  - T= terminal time step (the step where the episode ends)
- First-Visit Monte Carlo Update
  - During the backward update process, a state–action pair (s, a) is updated ONLY the **first time** it is encountered in **THIS episode**.
  - If this is the first visit of the state–action pair (s, a) in the episode:
    - **Q-value update**: $Q(s,a) \leftarrow Q(s,a) + \frac{G_t - Q(s,a)}{N(s,a)}$
    - $\boldsymbol{N(s,a)}$: the total number of times the state–action pair (s, a) has been visited during training **across all episodes**.
- **Policy update**: $\pi(s) = \arg\max_a \; Q(s,a)$
- **Epsilon update** every 100 episodes.
- Tracks **win rates** and **learning progress**.

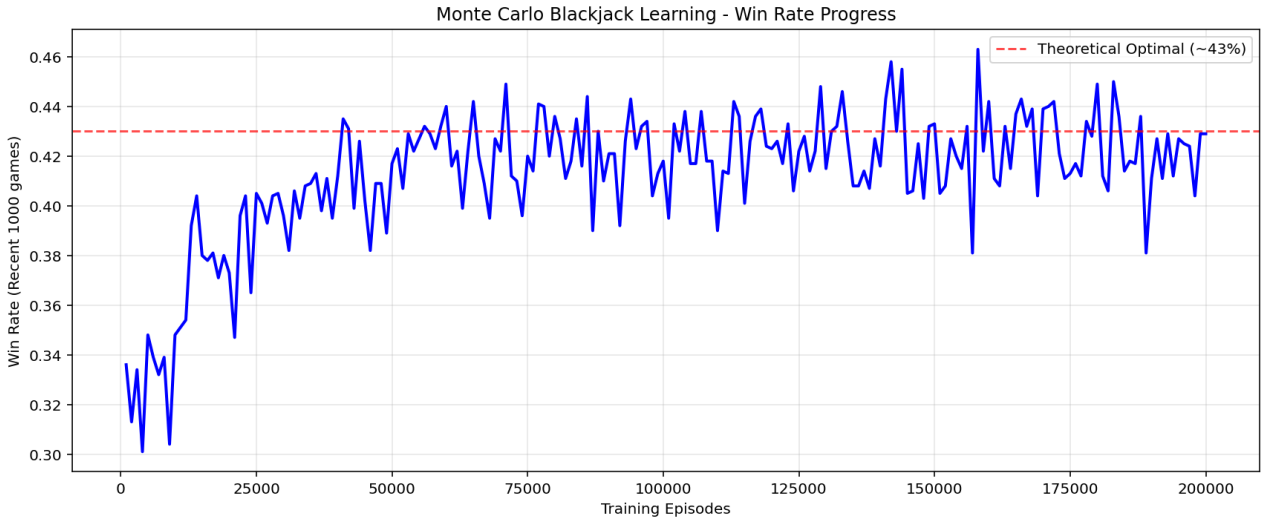
Performance Evaluation

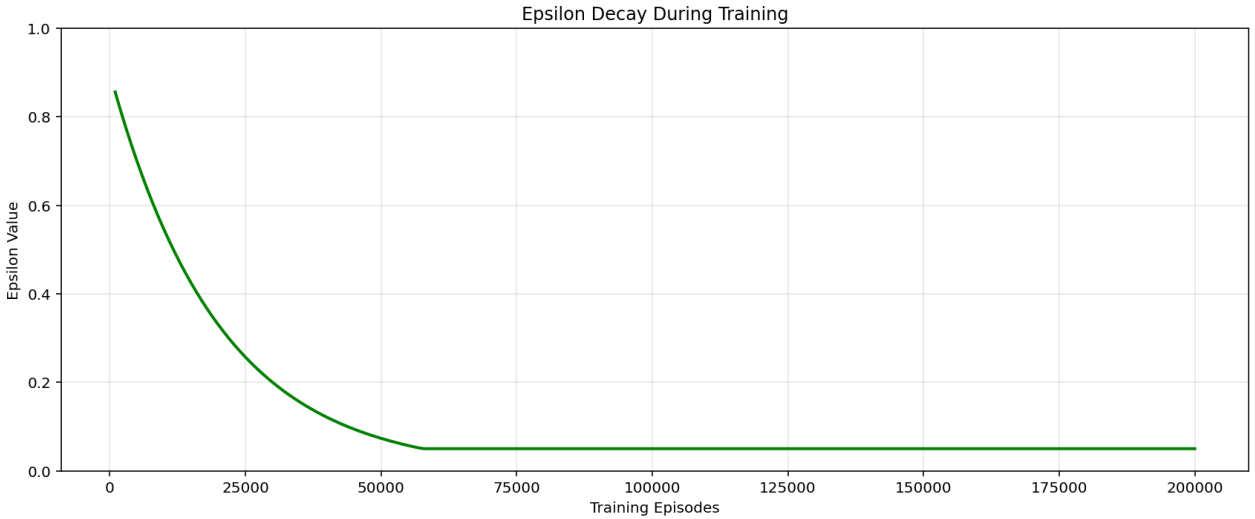
- **10,000 test games** after training.
- **Pure exploitation** during testing.
- Collects **win-loss-tie statistics**.
- Compares with **basic strategy** performance.
- Calculates **improvement percentage**.
- Records **learned states** count.
- Demonstrates **sample games**.

Results and Analysis

- **Basic strategy** achieves 42-44% win rate.
- **Monte Carlo strategy** aims for **near-optimal** performance.
- **Convergence analysis** confirms strategy stability.
- **Strategy table** shows learned decisions.
- **Visualization** of training progress.
- **Epsilon decay curves** show exploration evolution.

Results and Analysis





Reference

- https://zh.wikipedia.org/zh-tw/%E4%BA%8C%E5%8D%81%E4%B8%80%E9%BB%9E
- M. Lapan, *Deep Reinforcement Learning Hands-On*, 2nd ed. Birmingham, UK: Packt Publishing, 2020.
- A. Zai and B. Brown, *Deep Reinforcement Learning in Action*. Shelter Island, NY: Manning Publications, 2020.
- S. Ravichandiran, *Deep Reinforcement Learning with Python*, 2nd ed. Birmingham, UK: Packt Publishing, 2020.